Deep Learning

Lab 5

Value-based Reinforcement Learning

DQN, Atari Pong, and Enhanced DQN

Name : 陳映宬
Department : Electrical Engineering
Student ID : B11107027

May 2026

Contents

1 Introduction **2**

2 Implementation **3**
2.1 Required Implementation Questions . . . 3
2.2 Shared DQN Components . . . 4
2.3 Task 1: CartPole Vanilla DQN . . . 4
2.4 Task 2: Atari Pong Vanilla DQN . . . 5
2.5 Task 3: Enhanced DQN . . . 6
2.5.1 Double DQN . . . 6
2.5.2 Prioritized Experience Replay . . . 7
2.5.3 Multi-step Return . . . 7
2.5.4 Dueling Network . . . 7

3 Experimental Setup and Reproducibility **9**
3.1 Environment . . . 9
3.2 Training Commands . . . 9
3.3 Evaluation Commands . . . 10
3.4 Submitted Checkpoints . . . 11

4 Analysis and Discussion **12**
4.1 Task 1 Result Analysis . . . 12
4.2 Task 2 Result Analysis . . . 13
4.3 Task 3 Main Result . . . 14
4.4 Ablation Study . . . 18

5 Bonus **20**
5.1 Dueling Network . . . 20
5.2 Optimizer and Exploration Tuning . . . 20

6 Conclusion **25**

1 Introduction

This lab studies value-based reinforcement learning through DQN and its enhanced variants. Although the basic DQN framework is conceptually simple, the main challenge in this lab was not only to implement the algorithm, but also to make training stable and sample-efficient under a fixed environment step budget.

Across the three tasks, I found that CartPole can be solved reliably with a vanilla MLP-based DQN, while Atari Pong exposes the practical difficulties of DQN training: sparse rewards, high variance across evaluation seeds, and checkpoint drift during long training. In Task 2, vanilla DQN can eventually learn a strong Pong policy, but it requires many environment interactions. In Task 3, the goal is therefore to improve sample efficiency and reach high evaluation scores earlier.

The most important finding is that no single enhancement was sufficient by itself. Double DQN, PER, multi-step return, and the dueling architecture all contributed to better performance, but the final stability depended heavily on the training recipe, including the epsilon schedule, Adam epsilon, target network update frequency, and learning-rate decay. In particular, applying a learning-rate decay after 600k environment steps helped prevent policy drift and kept the later milestone checkpoints above the required score threshold.

Key Findings:

- **Task 1:** CartPole is solved reliably by vanilla DQN, reaching an average reward of **500.00** over 20 testing seeds.
- **Task 2:** Vanilla DQN can solve Pong eventually, reaching an average reward of **19.05**, but the training is less sample-efficient than the enhanced Task 3 agent.
- **Task 3:** The enhanced agent reaches an average reward of **19.10 at 600k** environment steps and stays above 19 at all later required milestones.
- The final result comes from the combined recipe: **Dueling DDQN + PER + 3-step return + exponential epsilon decay + learning-rate decay**.
- Ablation experiments show that **removing Double DQN causes the largest drop** among the required enhancements, while PER and 3-step return also improve sample efficiency. Some intuitive changes, such as longer n-step returns or NoopReset, did not consistently help.

2 Implementation

2.1 Required Implementation Questions

This subsection directly answers the required implementation questions from the lab description. For each item, I first describe the concept and then summarize how it is implemented in `dqn.py`.

How do I obtain the Bellman error for DQN?

For each sampled transition, DQN compares the current estimate $Q_{\text{online}}(s, a)$ with a bootstrapped target

$$y = r + \gamma(1 - d) \max_{a'} Q_{\text{target}}(s', a').$$

The Bellman error is therefore

$$\delta = y - Q_{\text{online}}(s, a).$$

In my implementation, the current Q-value is selected by `q_net(states).gather(...)` and the target is computed with the target network inside a `torch.no_grad()` block. The code stores `td_errors = target - q_values` and optimizes a Smooth L1 loss over this error.

How do I modify DQN to Double DQN?

Vanilla DQN uses the target network to both choose and evaluate the next action, which can overestimate Q-values. Double DQN separates these two roles: the online network selects

$$a^* = \arg\max_a Q_{\text{online}}(s', a),$$

and the target network evaluates $Q_{\text{target}}(s', a^*)$. In the code, this behavior is enabled by `--use-double`: `q_net` computes `next_actions`, and `target_net(...).gather(...)` obtains the corresponding target Q-values.

How do I implement the memory buffer for PER?

Prioritized Experience Replay samples transitions with probability proportional to their priority, where high-TD-error transitions are considered more informative:

$$P(i) = \frac{p_i^\alpha}{\sum_k p_k^\alpha}, \qquad p_i = |\delta_i| + \epsilon.$$

To reduce the bias from non-uniform sampling, the sampled loss is weighted by normalized importance-sampling weights. My `PrioritizedReplayBuffer` stores Atari states, actions, rewards, next states, done flags, and discounts in fixed NumPy arrays. It uses a `SumTree` for efficient priority sampling, returns indices and IS weights from `sample()`, and refreshes priorities after each update through `update_priorities()`.

How do I modify the 1-step return to multi-step return?

The 1-step target uses only the immediate reward before bootstrapping. In multi-step learning, I accumulate rewards over n steps:

$$R_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k r_{t+k}.$$

The replay transition then stores

$$(s_t,\ a_t,\ R_t^{(n)},\ s_{t+n},\ d,\ \gamma^n).$$

In the implementation, `NStepWrapper` keeps recent transitions in a `deque`; when enough transitions are available, it pushes the aggregated n-step transition into the underlying replay buffer. The training target uses the stored discount value, so terminal transitions and shorter end-of-episode returns are handled correctly.

How do I use Weight & Biases to track model performance?

I use W&B to monitor both learning progress and training stability. The code initializes a run with `wandb.init()` and defines `Env Step Count` as the x-axis for all logged metrics. During training, I log total reward, evaluation reward, epsilon, update count, loss, Q-value statistics, target statistics, TD-error mean/max, replay size, gradient norm, and learning rate. These curves are used in the analysis section to compare vanilla DQN, enhanced DQN, and ablation runs under the same environment-step axis.

2.2 Shared DQN Components

The DQN agent learns an action-value function $Q(s, a; \theta)$ using the Bellman target. For vanilla DQN, the target value is

$$y = r + \gamma(1 - d) \max_{a'} Q_{\text{target}}(s', a'; \theta^-),$$

where d indicates whether the transition is terminal. The Bellman error is the difference between this target and the current Q-value estimate:

$$\delta = y - Q(s, a; \theta).$$

In the code, this value is stored as `td_errors`. The implementation optimizes a Smooth L1 (Huber) loss over the Bellman error, which behaves like mean squared error near zero but is less sensitive to large TD-error outliers:

$$\mathcal{L}(\theta) = \mathbb{E}\left[\text{Huber}(\delta)\right].$$

Experience replay is used to reduce temporal correlation between consecutive samples. Transitions are stored in a replay buffer and mini-batches are sampled for gradient updates. A target network is updated periodically by copying the online Q-network parameters, which stabilizes the bootstrapping target.

Exploration Strategy:

The agent uses an epsilon-greedy policy. With probability ϵ, the agent selects a random action; otherwise, it selects the action with the maximum predicted Q-value:

$$a_t = \begin{cases} \text{random action}, & \text{with probability } \epsilon, \\ \arg\max_a Q(s_t, a; \theta), & \text{otherwise.} \end{cases}$$

2.3 Task 1: CartPole Vanilla DQN

In Task 1, the objective is to keep the pole balanced for as long as possible. The CartPole observation consists of cart position, cart velocity, pole angle, and pole angular velocity. The action space contains two actions: moving the cart left or right.

Q-Network Architecture:

The CartPole agent uses a multi-layer perceptron (MLP) as the Q-network.

Table 1: Task 1 Q-network architecture.

Layer	**Output Size**	**Activation**
Input	4	–
Linear	128	ReLU
Linear	128	ReLU
Linear	2	–

Hyperparameters:

Table 2: Task 1 training hyperparameters.

Parameter	**Value**
Learning rate	1.0×10^{-3}
Batch size	64
Replay buffer size	50k
Discount factor γ	0.99
Epsilon schedule	exponential decay 0.995, $\epsilon_{\min} = 0.01$
Replay start size	1000
Target update frequency	200

2.4 Task 2: Atari Pong Vanilla DQN

Task 2 applies DQN to Atari Pong, where the input is a raw image frame. Since raw frames are high-dimensional and contain redundant visual information, each frame is preprocessed before being passed to the Q-network.

Atari Preprocessing:

The preprocessing pipeline includes:

- converting the frame to grayscale,
- resizing the frame to 84×84,
- stacking the most recent 4 frames as the state representation.

CNN Q-Network Architecture:

The Pong agent uses a convolutional Q-network. The input shape is $4 \times 84 \times 84$.

Table 3: Task 2 CNN Q-network architecture.

Layer	**Channels / Units**	**Kernel / Stride**	**Activation**
Conv2D	32	8×8 / 4	ReLU
Conv2D	64	4×4 / 2	ReLU
Conv2D	64	3×3 / 1	ReLU
Fully Connected	512	–	ReLU
Fully Connected	*[num actions]*	–	–

Hyperparameters:

Table 4: Task 2 training hyperparameters.

Parameter	**Value**
Learning rate	1.0×10^{-4}
Batch size	64
Replay buffer size	100k
Discount factor γ	0.99
Epsilon schedule	exponential decay 0.999995, $\epsilon_{\min} = 0.05$
Replay start size	50k
Target update frequency	5000
Parallel environments	4

W&B Tracking:

Weights & Biases is used to track training curves and diagnostic statistics. The main logged metrics include total reward, evaluation reward, environment step count, epsilon, update count, and optional debugging statistics such as action distribution.

2.5 Task 3: Enhanced DQN

Task 3 improves the Pong agent using several DQN enhancements. The goal is to increase sample efficiency and reach a high Pong evaluation score with fewer environment steps.

2.5.1 Double DQN

Vanilla DQN may overestimate Q-values because the same network is used for both action selection and action evaluation. Double DQN separates these two roles:

$$a^* = \arg\max_{a'} Q_{\text{online}}(s', a'; \theta),$$

$$y_{\text{DDQN}} = r + \gamma(1 - d) Q_{\text{target}}(s', a^*; \theta^-).$$

The online network selects the next action, while the target network evaluates that selected action.

2.5.2 Prioritized Experience Replay

Prioritized Experience Replay samples more informative transitions more often. Each transition is assigned a priority based on the temporal-difference error:

$$p_i = |\delta_i| + \varepsilon_{\text{PER}}.$$

Here, ε_{PER} is a small positive constant used only for priority smoothing, and is different from the exploration epsilon in the epsilon-greedy policy. The probability of sampling transition i is

$$P(i) = \frac{p_i^\alpha}{\sum_k p_k^\alpha},$$

where α controls how strongly prioritization is used.

To correct the bias introduced by non-uniform sampling, importance sampling weights are applied:

$$w_i = \left(\frac{1}{N \cdot P(i)}\right)^\beta,$$

where β is annealed during training. In the implementation, the PER buffer supports `add()`, `sample()`, and `update_priorities()`.

2.5.3 Multi-step Return

The n-step return propagates rewards over multiple future steps:

$$R_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k r_{t+k} + \gamma^n \max_a Q(s_{t+n}, a).$$

Compared with 1-step bootstrapping, multi-step return can improve credit assignment in sparse-reward environments such as Pong. In this lab, the best current setting uses $n = 3$.

2.5.4 Dueling Network

The dueling architecture decomposes the Q-value into a state-value stream and an advantage stream:

$$Q(s, a) = V(s) + \left(A(s, a) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a')\right).$$

This helps the network estimate the value of states even when the choice of action has only a small immediate effect.

Final Configuration:

Table 5: Task 3 final configuration.

Parameter	**Value**
Algorithm	Dueling Double DQN + PER + 3-step return
Learning rate	1.8×10^{-4}
Adam epsilon	1.5×10^{-4}
LR decay	$\mathbf{1.8 \times 10^{-4} \rightarrow 1.0 \times 10^{-5}}$ **at 600k steps**
Replay buffer size	200k
Batch size	32
Train per environment step	1
Discount factor γ	0.99
Epsilon decay type	Exponential
Epsilon decay rate	0.99996
Epsilon decay steps	260k
Minimum epsilon	0.01
PER α	0.6
PER β (initial)	0.4
PER β anneal steps	600k
Replay start size	20k
Target update frequency	2000
Soft target update	disabled
NoopReset	0

3 Experimental Setup and Reproducibility

All reported scores are evaluated using 20 consecutive testing seeds. Task 1 uses seeds $0, \ldots, 19$, Task 2 uses seeds $26, \ldots, 45$, and Task 3 uses seeds $2, \ldots, 21$. The report includes evaluation screenshots because missing screenshots may lead to grading penalties.

3.1 Environment

Table 6: Experiment environment.

Item	Value
Python version	3.12.11 (Anaconda, GCC 11.2.0)
PyTorch version	2.8.0+cu128
Gymnasium version	1.1.1
ALE version	0.11.2
Hardware / platform	Lightning AI on Linux 6.8.0 AWS, Tesla T4, CUDA enabled

3.2 Training Commands

Task 1:

```
LAB5_DIR=/teamspace/studios/this_studio/lab5
SAVE_DIR="$LAB5_DIR/results_task1_lighting_ai_cartpole_vanilla"
WANDB_RUN=task1-lighting_ai-cartpole-vanilla

python -u dqn.py \
    --task 1 \
    --student-id B11107027 \
    --env-name CartPole-v1 \
    --seed 42 \
    --episodes 1500 \
    --save-dir "$SAVE_DIR" \
    --wandb-run-name "$WANDB_RUN" \
    --batch-size 64 \
    --memory-size 50000 \
    --lr 0.001 \
    --epsilon-decay-type exp \
    --epsilon-decay 0.995 \
    --epsilon-min 0.01 \
    --replay-start-size 1000 \
    --target-update-frequency 200
```

Task 2:

```
LAB5_DIR=/teamspace/studios/this_studio/lab5
SAVE_DIR="$LAB5_DIR/results_task2_lighting_ai_pong_vanilla"
WANDB_RUN=task2-lighting_ai-pong-vanilla

python -u dqn.py \
    --task 2 \
    --student-id B11107027 \
    --env-name ALE/Pong-v5 \
```

```
    --seed 42 \
    --episodes 10000 \
    --save-dir "$SAVE_DIR" \
    --wandb-run-name "$WANDB_RUN" \
    --batch-size 64 \
    --memory-size 100000 \
    --lr 0.0001 \
    --epsilon-decay-type exp \
    --epsilon-decay 0.999995 \
    --epsilon-min 0.05 \
    --replay-start-size 50000 \
    --target-update-frequency 5000 \
    --checkpoint-freq 50000
```

Task 3:

```
LAB5_DIR=/teamspace/studios/this_studio/lab5
SAVE_DIR="$LAB5_DIR/results_task3_lighting_ai_dueling_ref_adameps_\
nstep3_exp99996_lr18e5_decay600k_lr1e5_2500k"
WANDB_RUN=task3-lighting_ai-dueling-ref-adameps-nstep3-exp99996-\
lr18e5-decay600k-lr1e5-2500k

DRIVE_CKPT_DIR="$LAB5_DIR/checkpoints" \
PYTHONUNBUFFERED=1 python -u dqn.py \
    --task 3 \
    --student-id B11107027 \
    --env-name ALE/Pong-v5 \
    --seed 42 \
    --episodes 6000 \
    --save-dir "$SAVE_DIR" \
    --wandb-run-name "$WANDB_RUN" \
    --batch-size 32 \
    --memory-size 200000 \
    --lr 0.00018 \
    --adam-eps 0.00015 \
    --lr-decay-step 600000 \
    --lr-after-decay 1e-05 \
    --epsilon-decay-type exp \
    --epsilon-decay 0.99996 \
    --epsilon-decay-steps 260000 \
    --epsilon-min 0.01 \
    --replay-start-size 20000 \
    --target-update-frequency 2000 \
    --soft-target-tau 0.0 \
    --train-per-step 1 \
    --checkpoint-freq 500000 \
    --noop-max 0 \
    --max-env-steps 2500000 \
    --n-step 3 \
    --per-alpha 0.6 \
    --per-beta 0.4 \
    --per-beta-anneal-steps 600000 \
    --use-per --use-double --use-dueling
```

3.3 Evaluation Commands

Task 1 Evaluation:

```
python test_model.py \
    --env-name CartPole-v1 \
    --model-path LAB5_B11107027_task1.pt \
    --output-dir eval_task1_videos \
    --episodes 20 \
    --seed 0
```

Task 2 Evaluation:

```
python test_model.py \
    --env-name ALE/Pong-v5 \
    --model-path LAB5_B11107027_task2.pt \
    --output-dir eval_task2_videos \
    --episodes 20 \
    --seed 26
```

Task 3 Evaluation:

```
for STEPS in 600000 1000000 1500000 2000000 2500000; do
    python test_model.py \
        --env-name ALE/Pong-v5 \
        --model-path LAB5_B11107027_task3_${STEPS}.pt \
        --output-dir eval_task3_videos/${STEPS} \
        --episodes 20 \
        --seed 2
done

python test_model.py \
    --env-name ALE/Pong-v5 \
    --model-path LAB5_B11107027_task3_best.pt \
    --output-dir eval_task3_videos/best \
    --episodes 20 \
    --seed 2
```

3.4 Submitted Checkpoints

Table 7: Submitted checkpoint list.

Task	Checkpoint	Status
Task 1	`LAB5_B11107027_task1.pt`	selected final run
Task 2	`LAB5_B11107027_task2.pt`	selected from `model_ep2800.pt`
Task 3	`LAB5_B11107027_task3_600000.pt`	selected final run
Task 3	`LAB5_B11107027_task3_1000000.pt`	selected final run
Task 3	`LAB5_B11107027_task3_1500000.pt`	selected final run
Task 3	`LAB5_B11107027_task3_2000000.pt`	selected final run
Task 3	`LAB5_B11107027_task3_2500000.pt`	selected final run
Task 3	`LAB5_B11107027_task3_best.pt`	copy of 1.5M checkpoint (avg 20.15)

4 Analysis and Discussion

4.1 Task 1 Result Analysis

Figure 1 shows the Task 1 training curve. The submitted snapshot is the best evaluation checkpoint during training, saved as `LAB5_B11107027_task1.pt`.

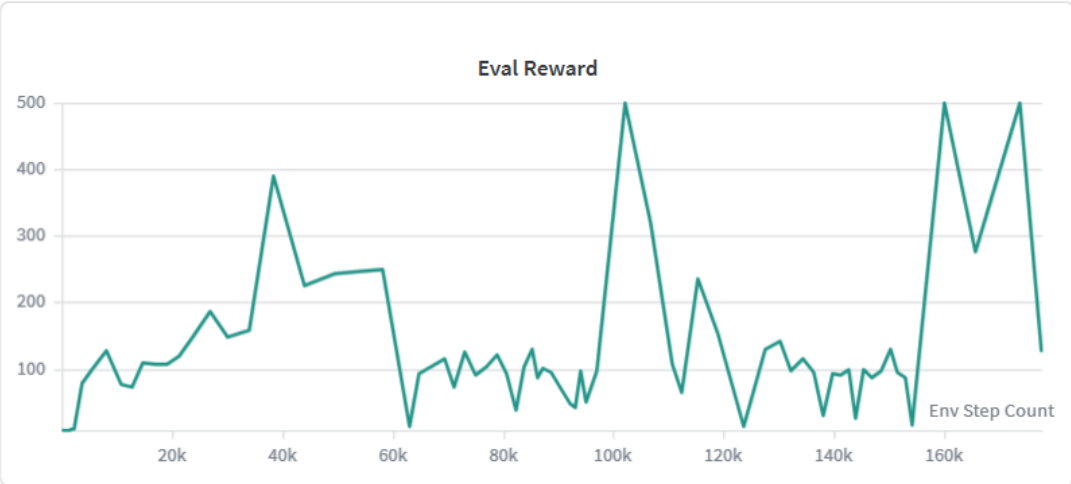
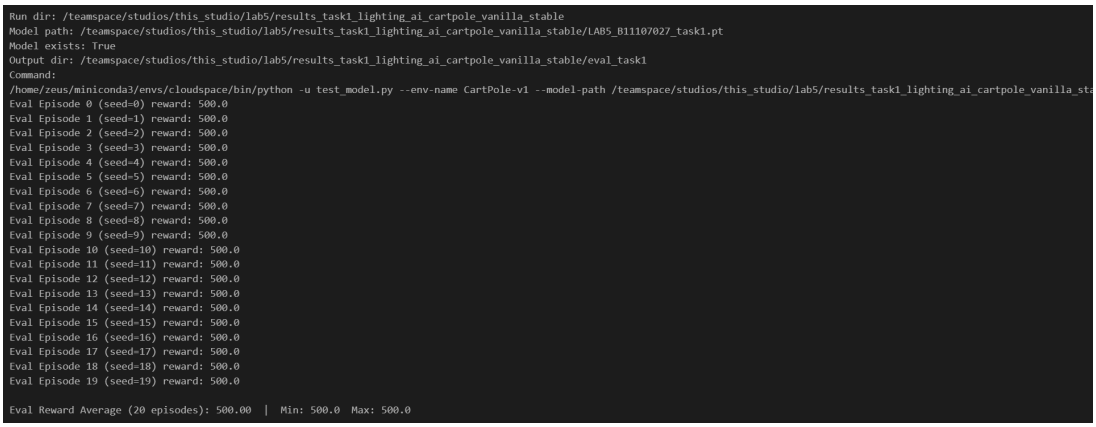


Figure 1: Task 1: episode reward vs. environment steps.



```
Run dir: /teamspace/studios/this_studio/lab5/results_task1_lighting_ai_cartpole_vanilla_stable
Model path: /teamspace/studios/this_studio/lab5/results_task1_lighting_ai_cartpole_vanilla_stable/LAB5_B11107027_task1.pt
Model exists: True
Output dir: /teamspace/studios/this_studio/lab5/results_task1_lighting_ai_cartpole_vanilla_stable/eval_task1
Command:
/home/zeus/miniconda3/envs/cloudspace/bin/python -u test_model.py --env-name CartPole-v1 --model-path /teamspace/studios/this_studio/lab5/results_task1_lighting_ai_cartpole_vanilla_sta
Eval Episode 0 (seed=0) reward: 500.0
Eval Episode 1 (seed=1) reward: 500.0
Eval Episode 2 (seed=2) reward: 500.0
Eval Episode 3 (seed=3) reward: 500.0
Eval Episode 4 (seed=4) reward: 500.0
Eval Episode 5 (seed=5) reward: 500.0
Eval Episode 6 (seed=6) reward: 500.0
Eval Episode 7 (seed=7) reward: 500.0
Eval Episode 8 (seed=8) reward: 500.0
Eval Episode 9 (seed=9) reward: 500.0
Eval Episode 10 (seed=10) reward: 500.0
Eval Episode 11 (seed=11) reward: 500.0
Eval Episode 12 (seed=12) reward: 500.0
Eval Episode 13 (seed=13) reward: 500.0
Eval Episode 14 (seed=14) reward: 500.0
Eval Episode 15 (seed=15) reward: 500.0
Eval Episode 16 (seed=16) reward: 500.0
Eval Episode 17 (seed=17) reward: 500.0
Eval Episode 18 (seed=18) reward: 500.0
Eval Episode 19 (seed=19) reward: 500.0

Eval Reward Average (20 episodes): 500.00  |  Min: 500.0  Max: 500.0
```

Figure 2: Task 1: 20-seed evaluation screenshot, seeds 0–19.

Evaluation Result:

$$\text{Average reward} = 500.00, \quad \text{Min} = 500.0, \quad \text{Max} = 500.0.$$

The evaluation uses 20 consecutive testing seeds from seed 0 to seed 19.

Discussion:

The final CartPole model consistently solves the environment. Every one of the 20 evaluation episodes reaches the maximum reward of 500, showing that the learned policy is robust across the selected testing seeds. During training, some intermediate fixed-episode snapshots were unstable, but the best-checkpoint selection correctly preserves the policy once it reaches a stable solution.

4.2 Task 2 Result Analysis

Figure 3 shows the Pong vanilla DQN training curve. The x-axis shows environment steps, as required by the assignment.

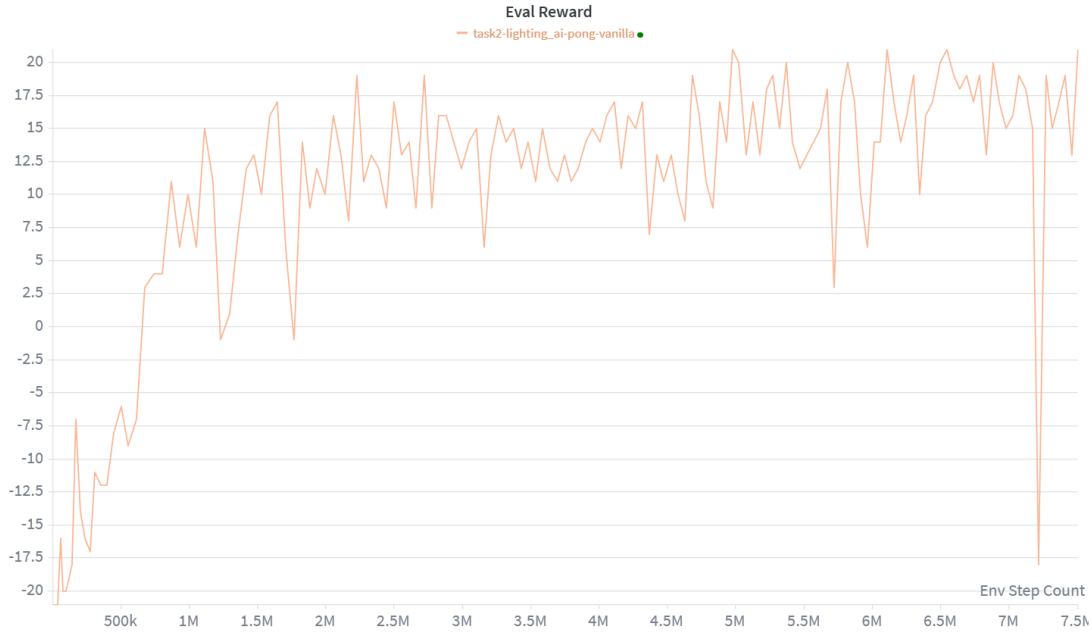


Figure 3: Task 2: evaluation reward vs. environment steps.

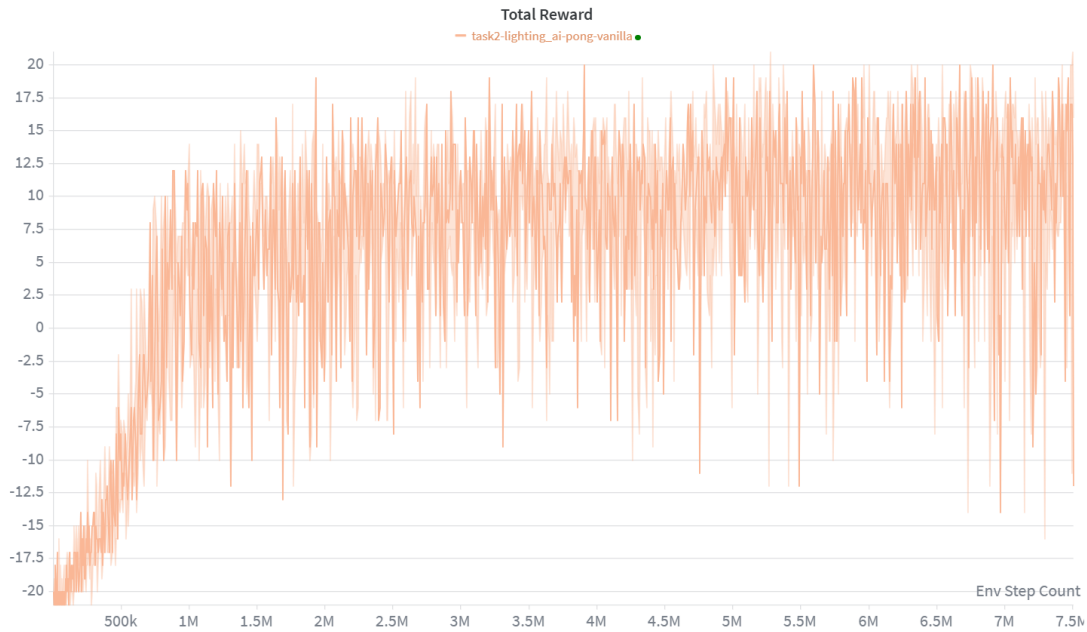
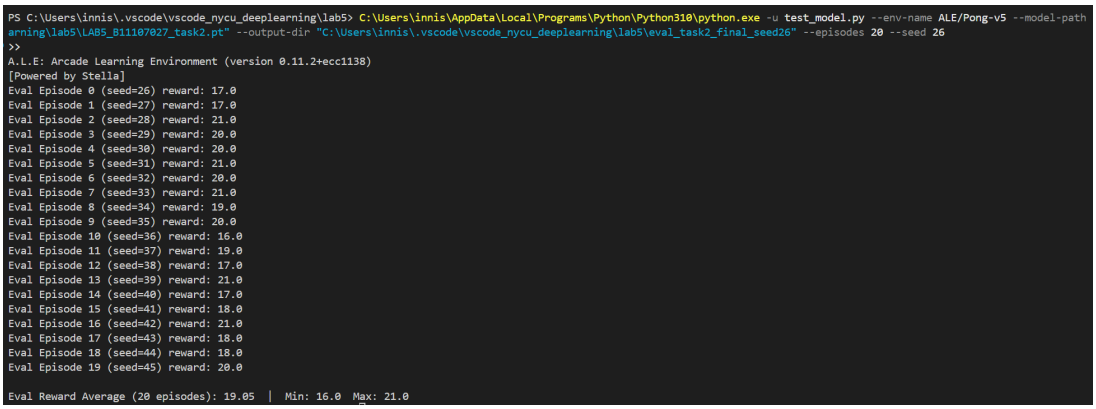


Figure 4: Task 2: total episodic reward vs. environment steps.



```
PS C:\Users\innis\.vscode\vscode_nycu_deeplearning\lab5> C:\Users\innis\AppData\Local\Programs\Python\Python310\python.exe -u test_model.py --env-name ALE/Pong-v5 --model-path
arning\lab5\LAB5_B11107027_task2.pt" --output-dir "C:\Users\innis\.vscode\vscode_nycu_deeplearning\lab5\eval_task2_final_seed26" --episodes 20 --seed 26
>>
A.L.E: Arcade Learning Environment (version 0.11.2+ecc1138)
[Powered by Stella]
Eval Episode 0 (seed=26) reward: 17.0
Eval Episode 1 (seed=27) reward: 17.0
Eval Episode 2 (seed=28) reward: 21.0
Eval Episode 3 (seed=29) reward: 20.0
Eval Episode 4 (seed=30) reward: 20.0
Eval Episode 5 (seed=31) reward: 21.0
Eval Episode 6 (seed=32) reward: 20.0
Eval Episode 7 (seed=33) reward: 21.0
Eval Episode 8 (seed=34) reward: 19.0
Eval Episode 9 (seed=35) reward: 20.0
Eval Episode 10 (seed=36) reward: 16.0
Eval Episode 11 (seed=37) reward: 19.0
Eval Episode 12 (seed=38) reward: 17.0
Eval Episode 13 (seed=39) reward: 21.0
Eval Episode 14 (seed=40) reward: 17.0
Eval Episode 15 (seed=41) reward: 18.0
Eval Episode 16 (seed=42) reward: 21.0
Eval Episode 17 (seed=43) reward: 18.0
Eval Episode 18 (seed=44) reward: 18.0
Eval Episode 19 (seed=45) reward: 20.0

Eval Reward Average (20 episodes): 19.05  |  Min: 16.0  Max: 21.0
```

Figure 5: Task 2: 20-seed evaluation screenshot, seeds 26–45.

Evaluation Result:

$$\text{Average reward} = 19.05, \quad \text{Min} = 16.0, \quad \text{Max} = 21.0.$$

Discussion:

The submitted Task 2 checkpoint is copied from `model_ep2800.pt`. It is a vanilla DQN Pong model trained with frame stacking, experience replay, a target network, and epsilon-greedy exploration. The training was less sample-efficient and less stable than the enhanced Task 3 agent because Pong has sparse rewards and vanilla DQN relies on one-step bootstrapping. However, after sufficient training and checkpoint selection, the model reaches the required threshold with an average reward of 19.05 over 20 consecutive seeds.

4.3 Task 3 Main Result

Task 3 compares the vanilla Pong DQN baseline with enhanced DQN variants. The main grading objective is to reach an average Pong score of 19 as early as possible among the required checkpoints.

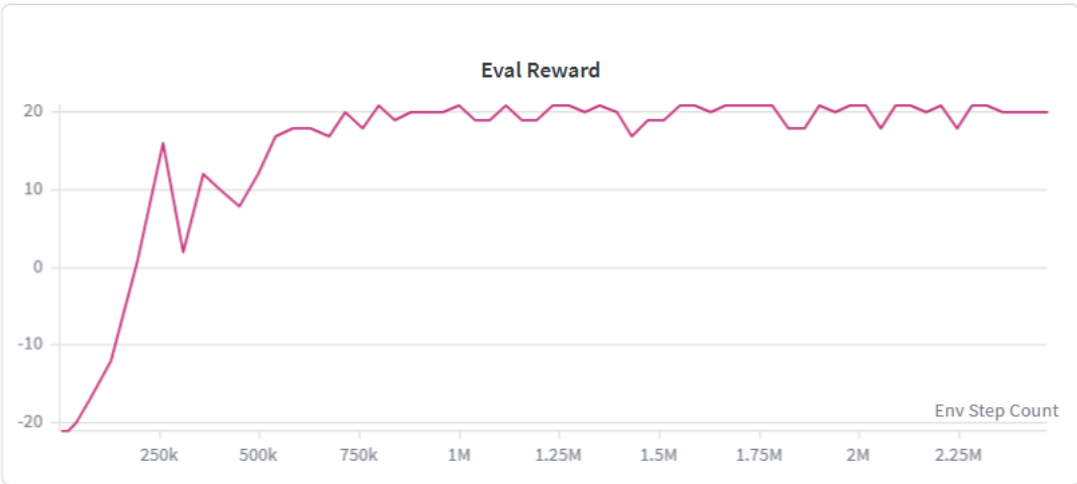


Figure 6: Task 3: final enhanced DQN evaluation reward curve.

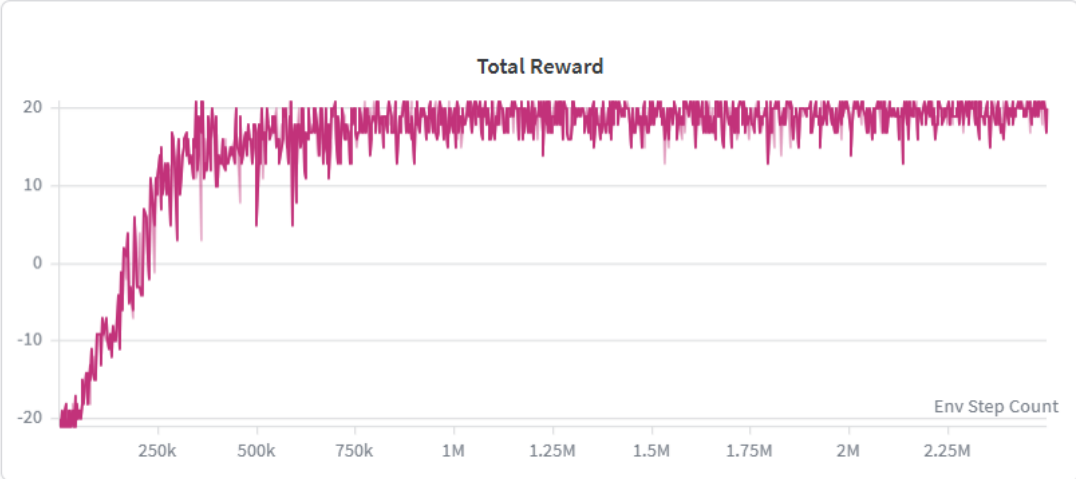


Figure 7: Task 3: final enhanced DQN training total reward curve.

Table 8: Task 3 milestone evaluation results.

Checkpoint	Average	Min	Max	Reached 19?
600k	**19.10**	14	21	Yes
1M	19.45	16	21	Yes
1.5M	**20.15**	18	21	Yes
2M	19.90	17	21	Yes
2.5M	19.80	18	21	Yes

```
=== Evaluating results_task3_lighting_ai_dueling_ref_adameps_nstep3_exp99996_lr18e5_decay600k_lr1e5_2500k @ 600000 steps ===
Using model: /teamspace/studios/this_studio/lab5/results_task3_lighting_ai_dueling_ref_adameps_nstep3_exp99996_lr18e5_decay600k_lr1e5_2500k/LAB5_B11107027_task3_600000.pt
Output dir: /teamspace/studios/this_studio/lab5/results_task3_lighting_ai_dueling_ref_adameps_nstep3_exp99996_lr18e5_decay600k_lr1e5_2500k/eval_task3_600000
A.L.E: Arcade Learning Environment (version 0.11.2+ecc1138)
[Powered by Stella]
Eval Episode 0 (seed=2) reward: 14.0
Eval Episode 1 (seed=3) reward: 21.0
Eval Episode 2 (seed=4) reward: 21.0
Eval Episode 3 (seed=5) reward: 16.0
Eval Episode 4 (seed=6) reward: 20.0
Eval Episode 5 (seed=7) reward: 20.0
Eval Episode 6 (seed=8) reward: 15.0
Eval Episode 7 (seed=9) reward: 20.0
Eval Episode 8 (seed=10) reward: 20.0
Eval Episode 9 (seed=11) reward: 20.0
Eval Episode 10 (seed=12) reward: 18.0
Eval Episode 11 (seed=13) reward: 21.0
Eval Episode 12 (seed=14) reward: 19.0
Eval Episode 13 (seed=15) reward: 20.0
Eval Episode 14 (seed=16) reward: 20.0
Eval Episode 15 (seed=17) reward: 21.0
Eval Episode 16 (seed=18) reward: 18.0
Eval Episode 17 (seed=19) reward: 20.0
Eval Episode 18 (seed=20) reward: 20.0
Eval Episode 19 (seed=21) reward: 18.0

Eval Reward Average (20 episodes): 19.10  |  Min: 14.0  Max: 21.0
```

Figure 8: Task 3: 600k checkpoint 20-seed evaluation screenshot, seeds 2–21.

```
=== Evaluating results_task3_lighting_ai_dueling_ref_adameps_nstep3_exp99996_lr18e5_decay600k_lr1e5_2500k @ 1000000 steps ===
Using model: /teamspace/studios/this_studio/lab5/results_task3_lighting_ai_dueling_ref_adameps_nstep3_exp99996_lr18e5_decay600k_lr1e5_2500k/LAB5_B11107027_task3_1000000.pt
Output dir: /teamspace/studios/this_studio/lab5/results_task3_lighting_ai_dueling_ref_adameps_nstep3_exp99996_lr18e5_decay600k_lr1e5_2500k/eval_task3_1000000
A.L.E: Arcade Learning Environment (version 0.11.2+ecc1138)
[Powered by Stella]
Eval Episode 0 (seed=2) reward: 21.0
Eval Episode 1 (seed=3) reward: 20.0
Eval Episode 2 (seed=4) reward: 20.0
Eval Episode 3 (seed=5) reward: 20.0
Eval Episode 4 (seed=6) reward: 20.0
Eval Episode 5 (seed=7) reward: 19.0
Eval Episode 6 (seed=8) reward: 21.0
Eval Episode 7 (seed=9) reward: 19.0
Eval Episode 8 (seed=10) reward: 20.0
Eval Episode 9 (seed=11) reward: 16.0
Eval Episode 10 (seed=12) reward: 16.0
Eval Episode 11 (seed=13) reward: 19.0
Eval Episode 12 (seed=14) reward: 21.0
Eval Episode 13 (seed=15) reward: 19.0
Eval Episode 14 (seed=16) reward: 18.0
Eval Episode 15 (seed=17) reward: 20.0
Eval Episode 16 (seed=18) reward: 20.0
Eval Episode 17 (seed=19) reward: 19.0
Eval Episode 18 (seed=20) reward: 20.0
Eval Episode 19 (seed=21) reward: 21.0

Eval Reward Average (20 episodes): 19.45  |  Min: 16.0  Max: 21.0
```

Figure 9: Task 3: 1M checkpoint 20-seed evaluation screenshot, seeds 2–21.

```
=== Evaluating results_task3_lighting_ai_dueling_ref_adameps_nstep3_exp99996_lr18e5_decay600k_lr1e5_2500k @ 1500000 steps ===
Using model: /teamspace/studios/this_studio/lab5/results_task3_lighting_ai_dueling_ref_adameps_nstep3_exp99996_lr18e5_decay600k_lr1e5_2500k/LAB5_B11107027_task3_1500000.pt
Output dir: /teamspace/studios/this_studio/lab5/results_task3_lighting_ai_dueling_ref_adameps_nstep3_exp99996_lr18e5_decay600k_lr1e5_2500k/eval_task3_1500000
A.L.E: Arcade Learning Environment (version 0.11.2+ecc1138)
[Powered by Stella]
Eval Episode 0 (seed=2) reward: 20.0
Eval Episode 1 (seed=3) reward: 21.0
Eval Episode 2 (seed=4) reward: 21.0
Eval Episode 3 (seed=5) reward: 20.0
Eval Episode 4 (seed=6) reward: 21.0
Eval Episode 5 (seed=7) reward: 19.0
Eval Episode 6 (seed=8) reward: 20.0
Eval Episode 7 (seed=9) reward: 20.0
Eval Episode 8 (seed=10) reward: 20.0
Eval Episode 9 (seed=11) reward: 21.0
Eval Episode 10 (seed=12) reward: 21.0
Eval Episode 11 (seed=13) reward: 20.0
Eval Episode 12 (seed=14) reward: 21.0
Eval Episode 13 (seed=15) reward: 20.0
Eval Episode 14 (seed=16) reward: 20.0
Eval Episode 15 (seed=17) reward: 18.0
Eval Episode 16 (seed=18) reward: 18.0
Eval Episode 17 (seed=19) reward: 21.0
Eval Episode 18 (seed=20) reward: 20.0
Eval Episode 19 (seed=21) reward: 21.0

Eval Reward Average (20 episodes): 20.15  |  Min: 18.0  Max: 21.0
```

Figure 10: Task 3: 1.5M checkpoint 20-seed evaluation screenshot, seeds 2–21.

```
=== Evaluating results_task3_lighting_ai_dueling_ref_adameps_nstep3_exp99996_lr18e5_decay600k_lr1e5_2500k @ 2000000 steps ===
Using model: /teamspace/studios/this_studio/lab5/results_task3_lighting_ai_dueling_ref_adameps_nstep3_exp99996_lr18e5_decay600k_lr1e5_2500k/LAB5_B11107027_task3_2000000.pt
Output dir: /teamspace/studios/this_studio/lab5/results_task3_lighting_ai_dueling_ref_adameps_nstep3_exp99996_lr18e5_decay600k_lr1e5_2500k/eval_task3_2000000
A.L.E: Arcade Learning Environment (version 0.11.2+ecc1138)
[Powered by Stella]
Eval Episode 0 (seed=2) reward: 21.0
Eval Episode 1 (seed=3) reward: 19.0
Eval Episode 2 (seed=4) reward: 21.0
Eval Episode 3 (seed=5) reward: 20.0
Eval Episode 4 (seed=6) reward: 20.0
Eval Episode 5 (seed=7) reward: 20.0
Eval Episode 6 (seed=8) reward: 21.0
Eval Episode 7 (seed=9) reward: 19.0
Eval Episode 8 (seed=10) reward: 20.0
Eval Episode 9 (seed=11) reward: 21.0
Eval Episode 10 (seed=12) reward: 21.0
Eval Episode 11 (seed=13) reward: 17.0
Eval Episode 12 (seed=14) reward: 20.0
Eval Episode 13 (seed=15) reward: 19.0
Eval Episode 14 (seed=16) reward: 19.0
Eval Episode 15 (seed=17) reward: 20.0
Eval Episode 16 (seed=18) reward: 19.0
Eval Episode 17 (seed=19) reward: 20.0
Eval Episode 18 (seed=20) reward: 20.0
Eval Episode 19 (seed=21) reward: 21.0

Eval Reward Average (20 episodes): 19.90  |  Min: 17.0  Max: 21.0
```

Figure 11: Task 3: 2M checkpoint 20-seed evaluation screenshot, seeds 2–21.

```
=== Evaluating results_task3_lighting_ai_dueling_ref_adameps_nstep3_exp99996_lr18e5_decay600k_lr1e5_2500k @ 2500000 steps ===
Using model: /teamspace/studios/this_studio/lab5/results_task3_lighting_ai_dueling_ref_adameps_nstep3_exp99996_lr18e5_decay600k_lr1e5_2500k/LAB5_B11107027_task3_2500000.pt
Output dir: /teamspace/studios/this_studio/lab5/results_task3_lighting_ai_dueling_ref_adameps_nstep3_exp99996_lr18e5_decay600k_lr1e5_2500k/eval_task3_2500000
A.L.E: Arcade Learning Environment (version 0.11.2+ecc1138)
[Powered by Stella]
Eval Episode 0 (seed=2) reward: 21.0
Eval Episode 1 (seed=3) reward: 21.0
Eval Episode 2 (seed=4) reward: 21.0
Eval Episode 3 (seed=5) reward: 20.0
Eval Episode 4 (seed=6) reward: 19.0
Eval Episode 5 (seed=7) reward: 19.0
Eval Episode 6 (seed=8) reward: 21.0
Eval Episode 7 (seed=9) reward: 21.0
Eval Episode 8 (seed=10) reward: 20.0
Eval Episode 9 (seed=11) reward: 21.0
Eval Episode 10 (seed=12) reward: 21.0
Eval Episode 11 (seed=13) reward: 21.0
Eval Episode 12 (seed=14) reward: 19.0
Eval Episode 13 (seed=15) reward: 20.0
Eval Episode 14 (seed=16) reward: 18.0
Eval Episode 15 (seed=17) reward: 18.0
Eval Episode 16 (seed=18) reward: 20.0
Eval Episode 17 (seed=19) reward: 19.0
Eval Episode 18 (seed=20) reward: 18.0
Eval Episode 19 (seed=21) reward: 18.0

Eval Reward Average (20 episodes): 19.80  |  Min: 18.0  Max: 21.0
```

Figure 12: Task 3: 2.5M checkpoint 20-seed evaluation screenshot, seeds 2–21.

Discussion:

The final Task 3 run reaches the required average score of 19 at the 600k checkpoint and stays above the threshold for all later required milestones. The main improvement comes from combining Dueling Double DQN, PER, and 3-step return with a carefully tuned exponential exploration schedule. The additional learning-rate decay after 600k steps is especially important: before 600k, the larger learning rate helps the agent learn Pong quickly; after 600k, reducing the learning rate to 1.0×10^{-5} prevents the policy from drifting away from the high-scoring solution during longer training.

Sample Efficiency: Vanilla vs. Enhanced DQN

Figure 13 compares the final enhanced run with two plain DDQN baselines. The stable plain DDQN baseline remains close to -21, indicating that the agent almost always loses the Pong game without scoring meaningful points. The conservative plain DDQN run only improves to around -7.5 by 600k steps. In contrast, the enhanced agent rapidly reaches high positive scores and remains near the 19–21 range during later training.

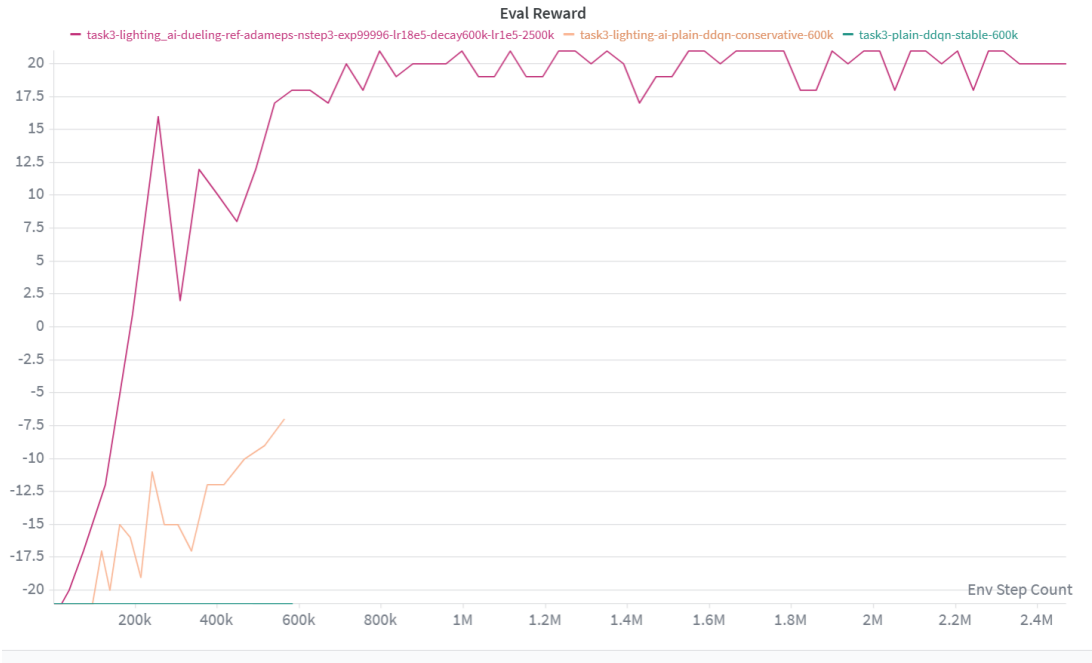


Figure 13: Task 3: sample efficiency comparison between plain DDQN baselines and the final enhanced DQN.

4.4 Ablation Study

This section focuses only on the three required Task 3 enhancements: Prioritized Experience Replay, Double DQN, and multi-step return. The comparisons use 20-seed evaluation at the 600k-step checkpoint. Table 9 shows that removing any required enhancement reduces the evaluation score. The largest drop appears when Double DQN is disabled, which supports the importance of reducing over-estimation bias in Pong. Removing PER or replacing the 3-step return with a 1-step return also lowers the score, showing that prioritized sample reuse and faster reward propagation both improve sample efficiency.

Table 9: Task 3 required 600k-step ablation summary. Each score is averaged over 20 consecutive evaluation seeds.

Run	Seed start	Average	Min	Max
Final enhanced agent	2	**19.10**	14.0	21.0
No PER	0	15.75	5.0	20.0
No Double DQN	0	**11.20**	7.0	17.0
No multi-step (1-step)	0	13.00	8.0	18.0

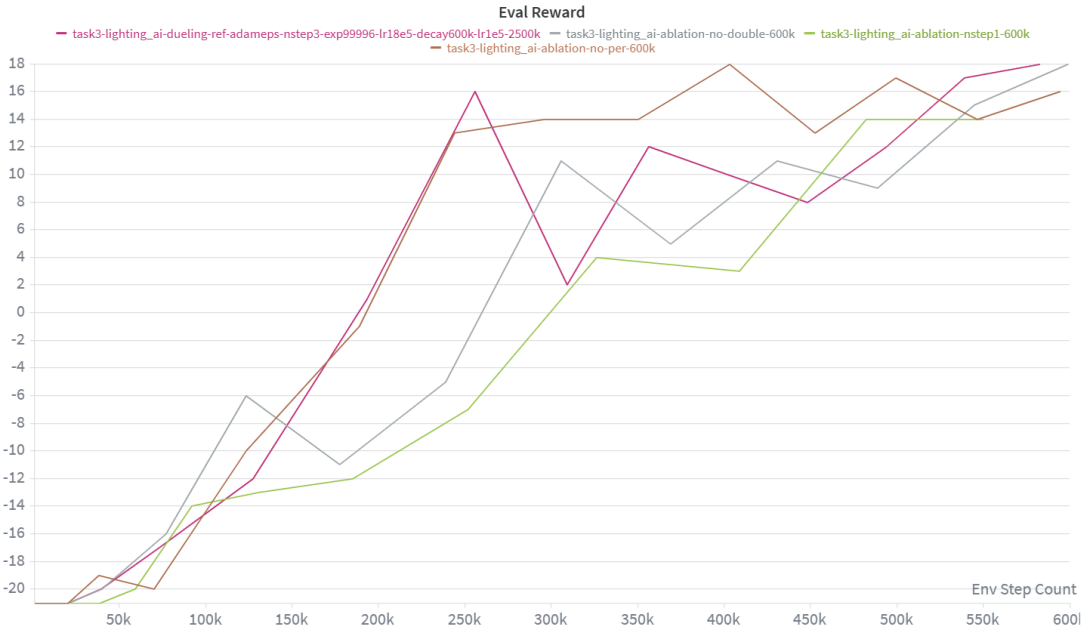


Figure 14: Task 3 required ablation: final agent vs. no PER, no Double DQN, and no multi-step variants.

5 Bonus

5.1 Dueling Network

The dueling architecture is included as an additional enhancement. It improves sample efficiency by separately estimating the value of a state and the relative advantage of each action. In the current experiments, adding the dueling architecture significantly improves the 600k-step evaluation result. Figure 15 compares the final dueling agent with a no-dueling variant. The no-dueling run improves, but its 600k evaluation score is only 12.80, far below the final agent's 19.10. This supports keeping the dueling architecture in the final Task 3 configuration.

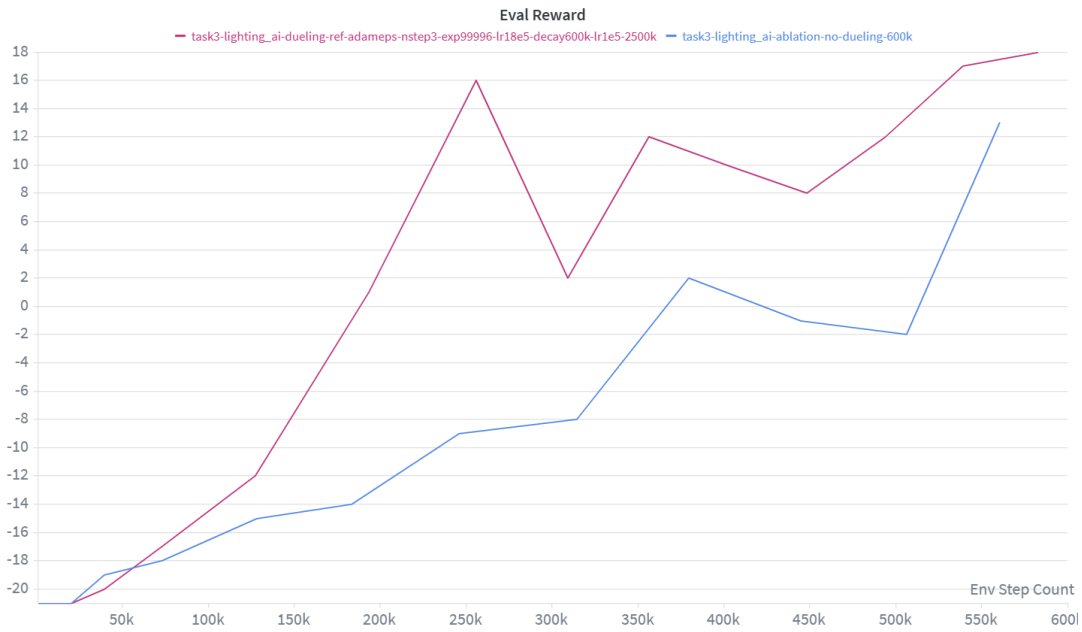


Figure 15: Bonus analysis: final dueling architecture vs. no-dueling variant.

5.2 Optimizer and Exploration Tuning

Additional experiments were conducted with Adam epsilon, exponential epsilon decay, NoopReset, replay warm-up length, learning-rate decay, soft target updates, and target update frequency. These experiments show that the training recipe is as important as the DQN enhancements themselves.

Table 10: Bonus strategy summary.

Strategy	Outcome	Interpretation
Dueling network	Helpful	Improves state-value estimation and seed robustness.
Adam epsilon tuning	Helpful	The reference-style optimizer recipe improved Pong stability.
Learning-rate decay after 600k	Helpful	Preserves the learned policy during long training.
5-step return	Not selected	Larger return horizon increased target variance.
NoopReset	Not selected	Did not consistently improve the current recipe.
Soft target update	Not selected	Did not beat hard target updates in final score.
Target update sweep	Diagnostic	Confirms that target update frequency affects stability.

Learning-rate Decay:

Figure 16 shows the most important stabilization experiment. Without **learning-rate decay**, the model can reach a high score around 600k steps but later becomes less stable. Decaying the learning rate from $\mathbf{1.8 \times 10^{-4}}$ to $\mathbf{1.0 \times 10^{-5}}$ **after 600k steps** preserves the learned policy and keeps the evaluation reward high through 2.5M steps.

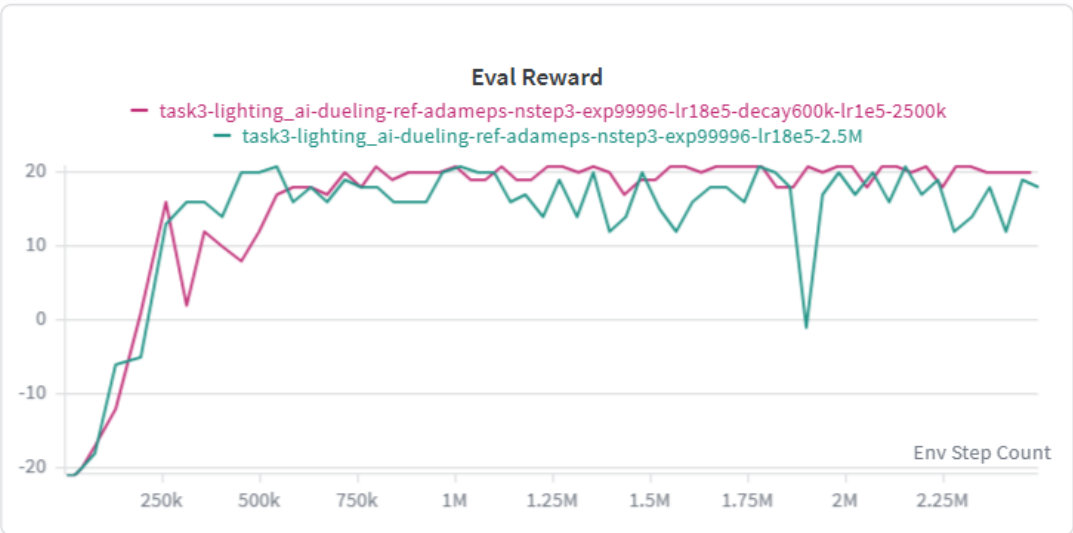


Figure 16: Bonus analysis: learning-rate decay stabilizes long training.

Multi-step Return Length:

Figure 17 compares 3-step and 5-step returns. A larger n can propagate sparse Pong rewards farther, but it also increases the variance of the target. In this experiment, $n = 5$ is less stable and remains well below the $n = 3$ run, so the final configuration uses $n = 3$.

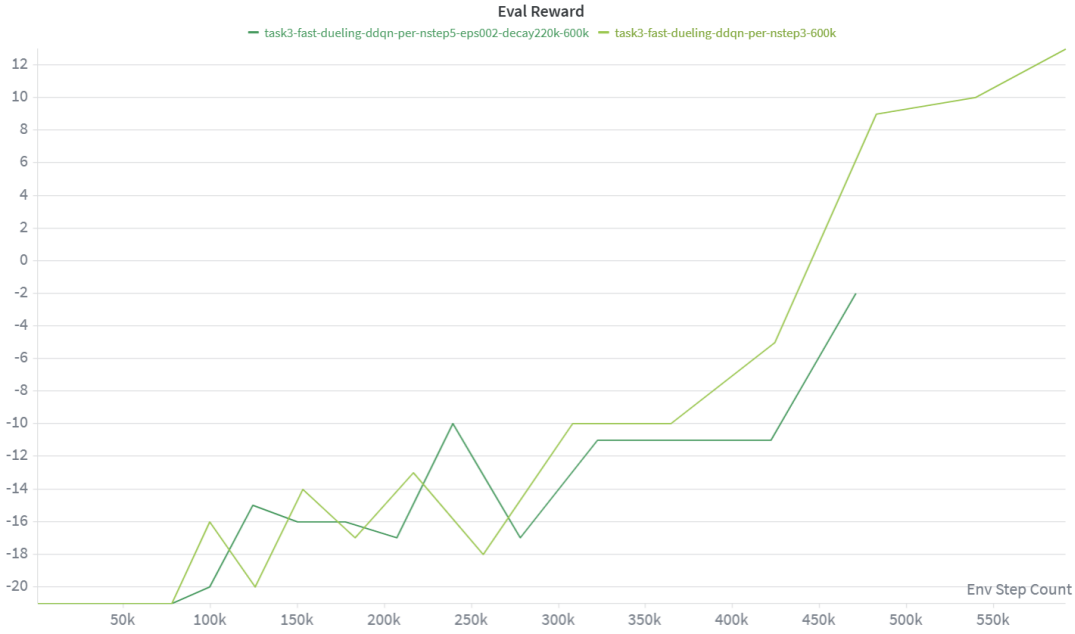


Figure 17: Bonus analysis: 3-step return vs. 5-step return.

Soft Target Update:

Figure 18 compares a soft target update variant with a hard target update baseline. Soft update improves the early curve in part of the run, but the hard-update baseline reaches a higher final evaluation score in this setting. Therefore, the final submitted model keeps hard target updates instead of using soft target update.

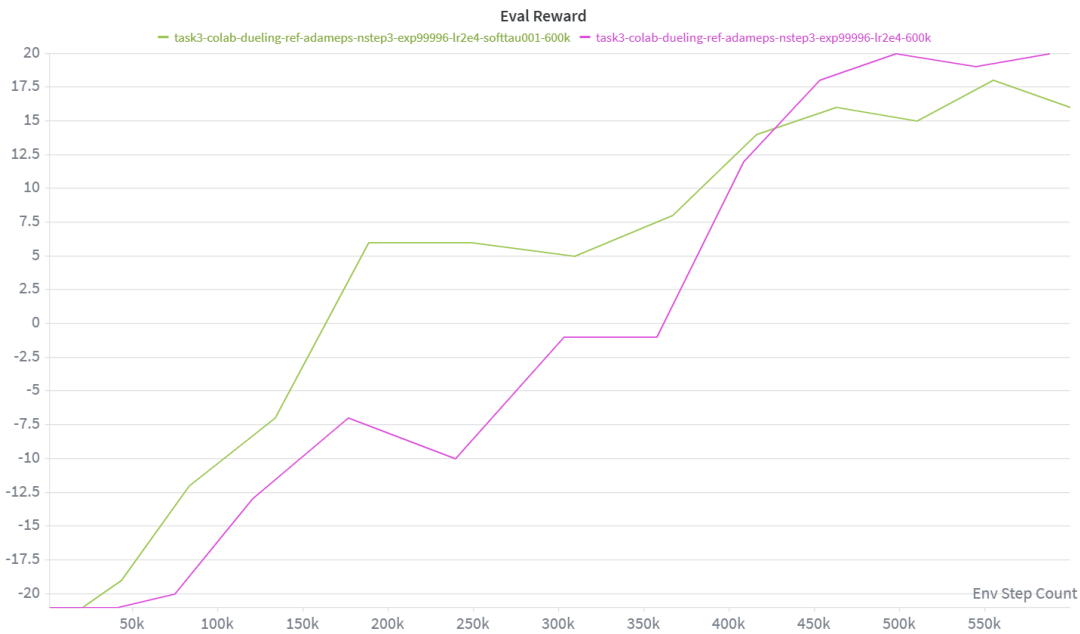


Figure 18: Bonus analysis: soft target update did not improve the final score.

Learning-rate Sensitivity:

Figure 19 shows that the training result is sensitive to the base learning rate. The run with 1.8×10^{-4} gives the best 600k behavior among the tested values. Slightly smaller or

larger learning rates can still learn Pong, but their curves are less reliable.

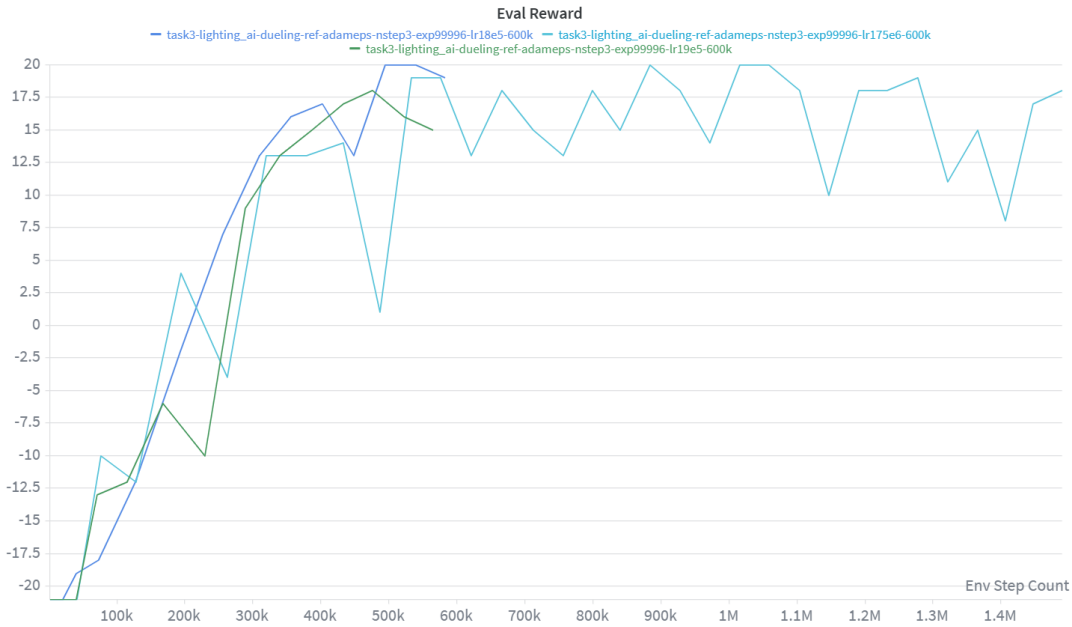


Figure 19: Bonus analysis: sensitivity to the base learning rate.

Target Update Frequency:

Figure 20 shows that target network update frequency also affects stability. The final recipe uses a hard target update every 2000 updates. More frequent updates can learn quickly but fluctuate more, while slower target updates learn less consistently in this set of experiments.

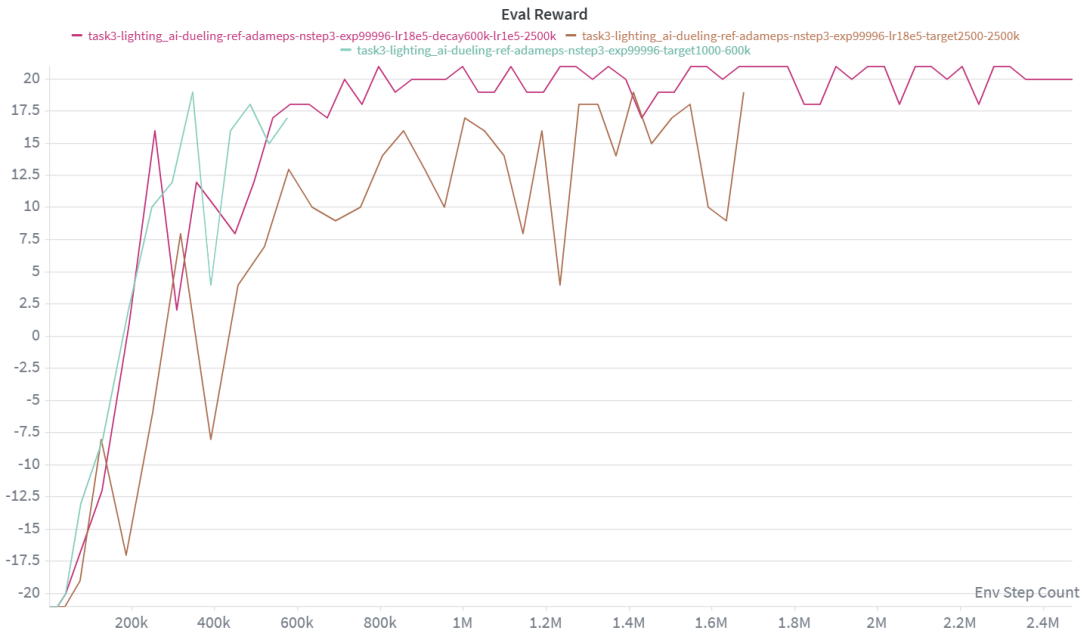


Figure 20: Bonus analysis: target update frequency sensitivity.

Summary:

Among the bonus-level strategies, the most useful additions are the dueling architecture, Adam epsilon tuning, and the post-600k learning-rate decay. Soft target update was also implemented and tested, but in the current setting it did not improve the final score compared with hard target updates. This negative result is still informative because it shows that the target network update mechanism is a sensitive part of the training recipe.

6 Conclusion

This lab implements vanilla DQN and several enhanced DQN variants for CartPole and Atari Pong. Task 1 demonstrates the basic DQN training pipeline on a simple low-dimensional control task. Task 2 extends the same framework to image-based Atari input with frame preprocessing and a CNN Q-network. Task 3 improves the Pong agent using Double DQN, PER, multi-step returns, and a dueling network.

The final Task 3 configuration reaches an average reward of **19.10 at 600k** environment steps and **remains above 19** at 1M, 1.5M, 2M, and 2.5M environment steps. The strongest empirical finding is that Dueling Double DQN with PER, 3-step return, a carefully tuned exponential epsilon decay, and **learning-rate decay after 600k steps** gives both strong sample efficiency and stable long-run performance.

Future Work:

Future experiments could focus on improving seed robustness and reducing the variance between different platforms. It would also be useful to test a smoother learning-rate schedule, such as cosine decay, and to explore a combined 1-step and multi-step loss. However, these changes require careful implementation because the replay buffer must preserve both short-horizon and multi-step transitions consistently.